czii-cryo-et-object-identification

4位解法 [ソースコードと提出Notebookを公開！]

Rank	4
Team	yu4u & tattaka
Author	yu4u
Topic ID	561401
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/czii-cryo-et-object-identification/discussion/561401
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

まず、このすばらしいコンペを開催してくださったコンペ主催者と Kaggle スタッフに感謝します。以下では、チーム yu4u & tattaka の解法を紹介します。

概要

私たちは、姿勢推定や顔のキーポイント検出で最も一般的に使われている手法である、ヒートマップベースの手法で粒子の点を検出するアプローチを採用しました。

このコンペでは 2D 画像ではなく 3D 画像を扱うので、3D ボクセルを入力として 3D ヒートマップを出力する、2種類の UNet 風のモデル（yu4u のモデルと tattaka のモデル）を使いました。

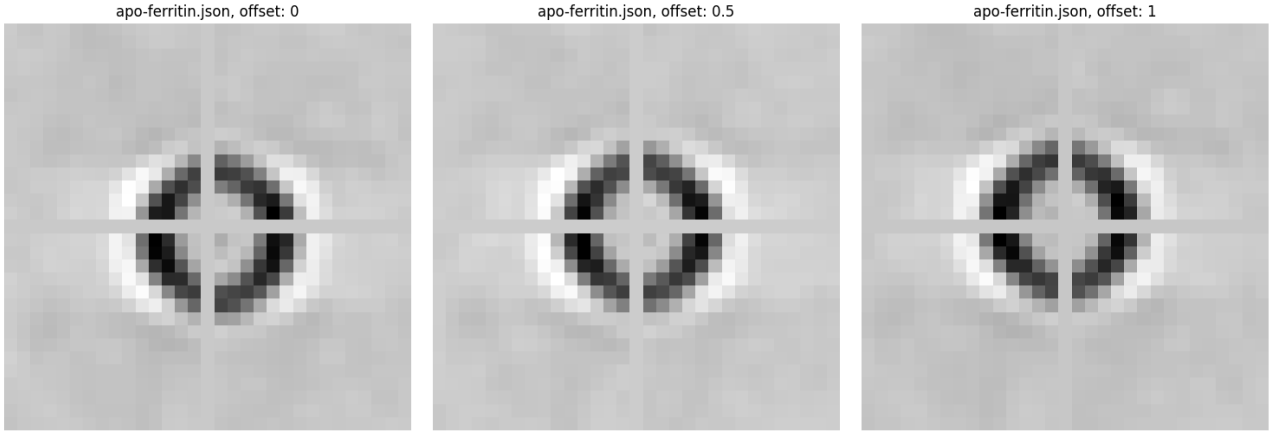
このコンペへの取り組み方

まず、このコンペへの取り組み方、特に CV と LB が相関しないという問題にどう対処したかを説明します。CV は、出力される指標がある程度妥当であることの確認と、チェックポイントの選択にだけ使いました。改善の見込みがあるモデルは単純に提出し、どの手法を採用・却下するかは LB に頼って決めました。

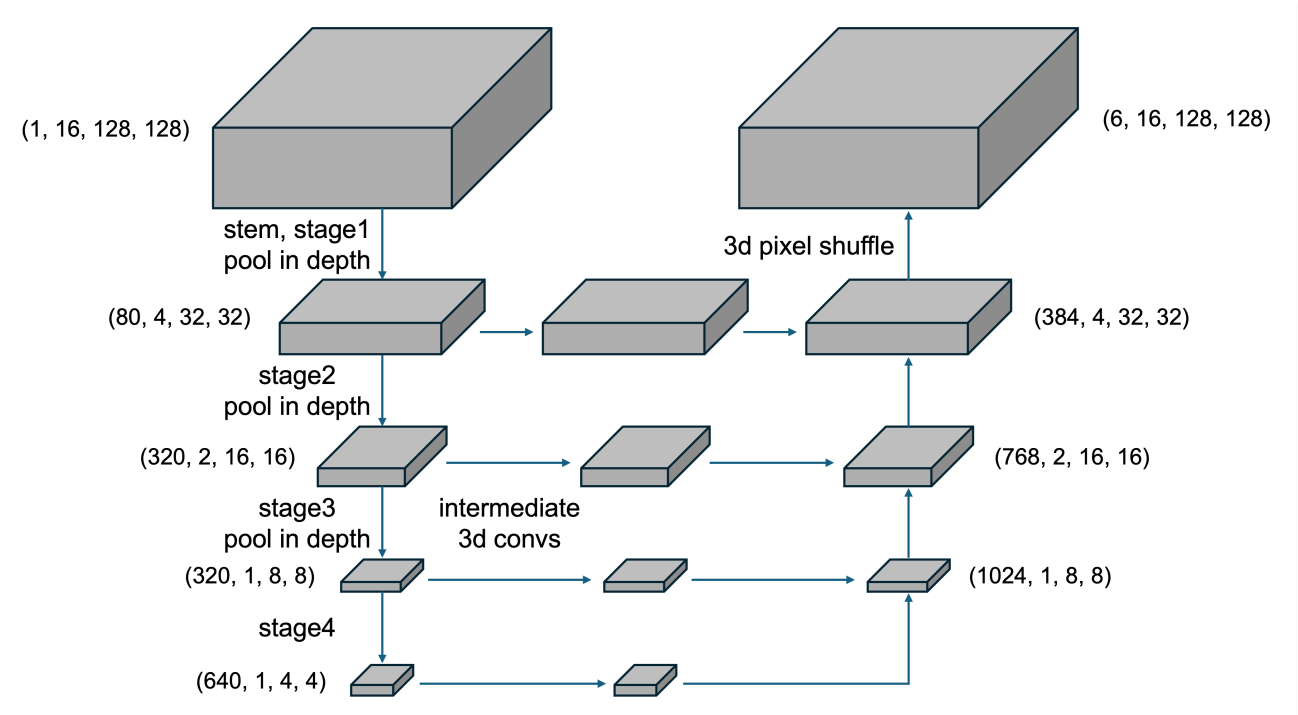
正解ヒートマップの作成

モデルの学習に必要な正解ヒートマップを生成します。正解の粒子座標をピクセル座標系に変換し、ガウス関数を使ってマスクを作ります。yu4u のモデルでは、粒子の中心を 1.0、sigma を 6 ピクセルにしています。tattaka のモデルでは、粒子の大きさに応じて粒子ごとに異なる sigma の値を使いました。

粒子座標をピクセル座標系に変換するときには、オフセット 1.0 を加えるべきだと考えています。この discussion では 0.5 を加えることが提案されていますが、私たちの notebook では 1.0 が正しい値であることを示しています。主な違いは、先の discussion が粒子の中心がピクセルの左上にあると仮定しているのに対し、私たちは、平均的には円はピクセルの中心 (0.5, 0.5) から描かれるべきだと主張している点です。



yu4u のモデル



2D 画像ベースのモデルを backbone に使う 2.5D-UNet を採用しました。この backbone の各ステージの出力は深さ方向にプーリングされ、深さ方向にも階層的な特徴抽出ができるようになっています。このアイデアはすばらしい notebook から借りました。興味深い観察として、このプーリング操作を stride 付きの 3D 畳み込みに置き換えると性能が下がりました。これは、プーリングによる方法が、元の 2D backbone の特徴マップをできるだけ保ったまま、深さ方向の特徴を効果的に集約するからでしょう。3D データを扱う他の多くの Kaggle コンペと同様に、2D backbone を使った UNet は、素直な 3D backbone の UNet を上回りました。

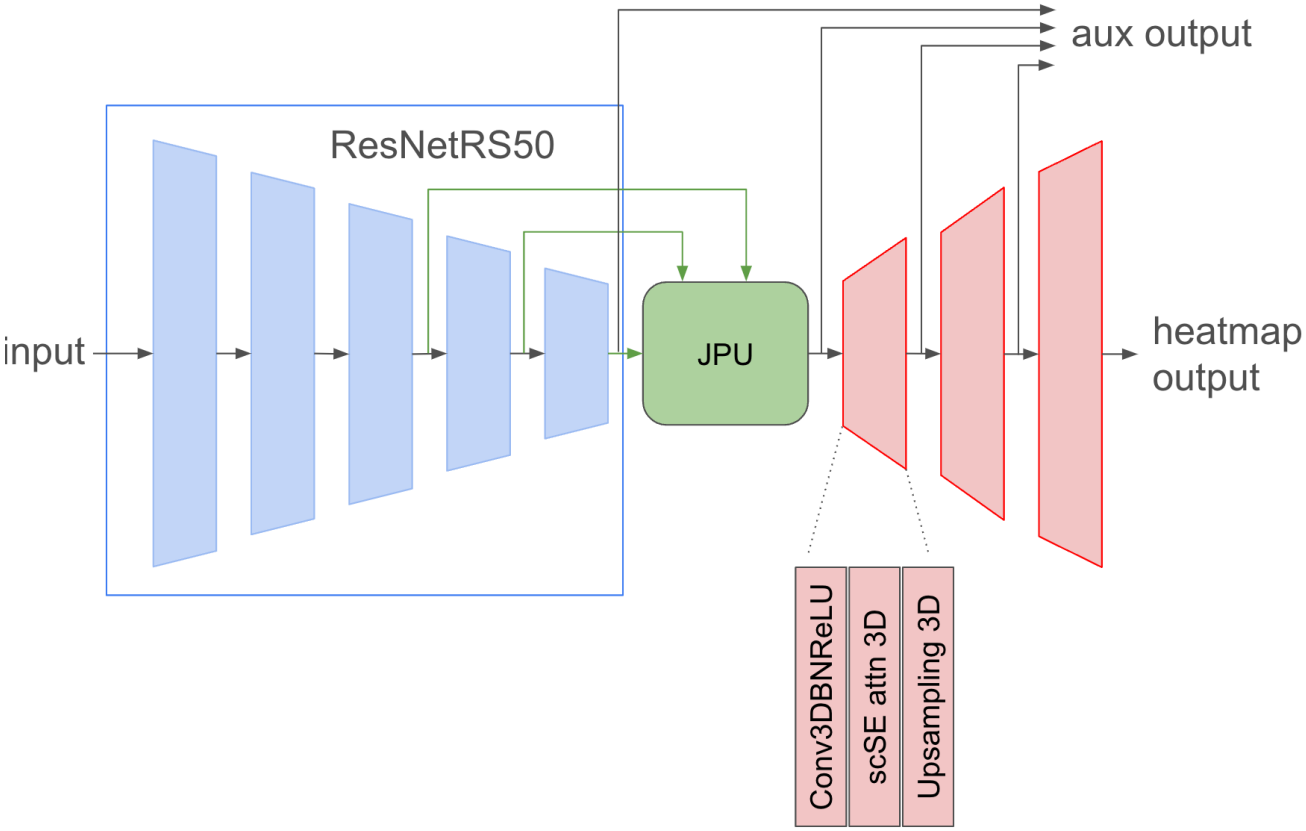
また、contrails コンペの3位解法にならって、エンコーダとデコーダの間に 3D 畳み込みを入れました。

最初は素の UNet アーキテクチャを使っていましたが、高解像度の特徴マップを処理するには大きなメモリと計算が必要でした。これに対処するため、stride 4 の特徴マップから pixel shuffle を使って最終的なヒートマップを出力するモデルを採用しました。Pixel shuffle（TensorFlow では depth_to_space とも呼ばれます）は、チャネル次元の情報を空間次元に再配置する操作です。deconvolution と比べて、計算効率とアーティファクトの低減の面で利点があります。

最終提出では、backbone に ConvNeXt Nano を使った、異なる fold の4つのモデルを使いました。

tattaka のモデル

モデルのアーキテクチャ



このモデルは、ResNetRS-50 を backbone にした軽量な 2.5D UNet です。

モデルの入力は 32×128×128（D×H×W）のボリュームで、同じサイズの 3D ヒートマップを出力します。backbone 内では、最初の2つのステージで average pooling を使って深さを段階的に半分にします。その後は、kernel=3、stride=1、padding=1 の average pooling を使って、深さを保ったまま特徴マップを計算します。その結果、backbone の各ステージでの特徴マップの形状は次のようになります。

(bs, ch, 16, 64, 64), (bs, ch, 8, 32, 32), (bs, ch, 8, 16, 16), (bs, ch, 8, 8, 8), (bs, ch, 8, 4, 4).

デコーダでは、解像度の低い3つの特徴マップを Joint Pyramid Upsampling に入力します。その後、3D CNN、SESC attention、アップサンプリング層を使って、入力ボリュームと同じサイズになるまで段階的にアップサンプリングします。

Loss 関数

ボリューム内の粒子数は比較的少ないので、学習中は陽性と陰性のサンプルの間に大きなクラス不均衡があります。正解ヒートマップを生成するパラメータを調整してみましたが、交差検証の性能は改善しませんでした。

最終的に、陽性と陰性のサンプルのバランスを取る単純な MSE ベースの loss 関数を実装しました。これにより収束が速くなりました。

```
loss = MeanSquaredError(pred, true)

pos_loss = (loss * true).() / (true.() + )
```

```
neg_loss = (loss * ( - true)).() / (( - true).() + )

balanced_loss = pos_loss + neg_loss
```

推論のコツ

最終提出では、yu4u のモデル4つと tattaka のモデル3つを使いました。

時間制限内に収めるため、モデルを TensorRT 形式に変換して推論を高速化しました。変換の手順はこの notebook に基づいています。

さらに、T4 GPU を2枚持つ Kaggle Notebook のインスタンスを選び、multiprocessing を使って推論を並列化しました。

後処理

最終的なヒートマップに対して、まず kernel size 7 の max pooling で実装した non-maximum suppression で局所最大を検出します。次に、検出した点を粒子の種類ごとに異なる閾値でフィルタします。

検出した点はピクセル座標系にあるので、粒子座標系に変換する必要があります。そのために、次の手順を踏みます。

- 中心化: ピクセル座標に 0.5 を加えて、ピクセルの左上から中心へずらす。
- オフセットの補正: ヒートマップ生成時に加えたオフセット 1.0 を引く。
- スケーリング: 補正したピクセル座標に 10.012 を掛けて、粒子座標系に変換する。

私たちには効かなかったこと

- 2段階モデル: ヒートマップ手法で検出した点の周りの領域を切り出し、その領域に分類モデルを適用してスコアを補正するモデルを作りました。CV スコアの面ではうまくいきましたが、LB の性能は改善しませんでした。

ソースコードと Notebook

- 最終提出
- tattaka のモデルの学習
- yu4u のモデルの学習

補足Q&A

suguuuuu — 質問

おめでとうございます！！！！！！

3つ質問があります。

- 標準的な MSE、Focal loss、BCE と比べて、あなたが使った loss 関数は性能に違った効果がありましたか（どのくらいの差ですか）。
- kernel size を 7 にしていますが、評価対象の粒子の大きさに応じて可変にしなかった理由はありますか。
- max pooling を使っていますが、average pooling を使わなかった理由はありますか。

tattaka — 回答

- 最初は Focal Loss、BCE、FBeta Loss で学習を試しましたが、最終的な精度は MSE ベースの loss の方が良くなりました。また、平滑化に EMA を使う場合は、陽性と陰性のサンプルのバランスを取ることが決定的に重要です。そうしないと収束が大幅に遅くなります。
- 主催者の notebook では、kernel size は粒子ごとに動的に調整されていました。同じアプローチの実装も検討しましたが、優先度が低く、提出回数も限られていたため、試せませんでした。
- 後処理の max pooling は、ピーク検出の仕組みとして機能しています。